



KCA UNIVERSITY

School of Computing and Information Technology

Department of Computing

WORK IN PROGRESS DOCUMENT

**TechPulse: Predicting Developer Technology Decline Using Community Signals and Enterprise Adoption Patterns ,
A Machine Learning Approach**

Unit Code & Title	STU 4101 – Final Year Project I
Program	Bachelor of Science in Data Science
Student Name	Kitili Tony Kenga
Registration Number	24/03652
Supervisor	Dr. Rufus Gireka
ORCID	0009-0007-6899-8590
Faculty	School of Technology (SoT)
Date Submitted	July 22, 2026
Document Version	1.0 (Work in Progress)

Contents

WORK IN PROGRESS DOCUMENT	1
CHAPTER ONE: INTRODUCTION.....	3
1. Background.....	3
2. Problem Statement.....	4
3. Objectives	5
4. Significance of Project.....	5
CHAPTER TWO: LITERATURE REVIEW AND REQUIREMENTS	6
2.1 Literature Review	6
2.2 System Requirements Specifications.....	8
Table 2.1 , Summary of the TechPulse Software Environment	9
Table 2.2 , Condensed Functional Requirements by Group	10
Table 2.3 , Candidate Models, Justification, and Evaluation Metrics (Proposal §3.4).....	10
Table 2.4 , Condensed Non-Functional Requirements	13
CHAPTER THREE: METHODOLOGY AND DESIGN	14
3.1 Data Collection Methods	14
3.2 Systems Design Specifications	14
Table 3.1 , TechPulse Modules and Responsibilities (SDS §5.3.2).....	16
20	
3.2.5 Deployment and Security Design.....	20
3.3 Test Plan	21
Table 3.2 , TechPulse Test Plan Summary	22
3.4 Implementation Plan.....	22
Table 3.3 , Project Risks and Mitigations (Proposal Appendix D)	25
REFERENCES	25
APPENDICES.....	26
Appendix I: Research Schedule.....	26
Table A.1 , Semester 1 Research Schedule.....	26
Table A.2 , Semester 2 Research Schedule.....	27
Appendix II: Project Budget.....	27
Table A.3 , TechPulse Project Budget (Proposal Appendix B).....	27
Appendix III: Data Collection Instrument (e.g. Questionnaire)	28
Table A.4 , Data Sources Used in Place of a Primary Data-Collection Instrument	28

CHAPTER ONE: INTRODUCTION

1. Background

Technology adoption decisions carry substantial financial and strategic risk for both organisations and individual developers. Organisations that commit engineering resources to a technology that is entering decline face costly, disruptive platform migrations, while developers who specialise in a technology heading toward obsolescence face narrowing job prospects and forced reskilling. In 2023 alone, organisations globally spent an estimated USD 4.7 trillion on information technology, and a significant proportion of that investment was allocated to software tools

and platforms whose long-term viability was uncertain (Gartner, 2023). Despite the scale of this spending, most technology stakeholders continue to rely on trend articles, social media commentary, and informal peer opinion rather than rigorous, evidence-based analysis when deciding where to invest.

The pace at which technologies mature, stabilise, and eventually decline has accelerated at the global level. Technologies such as CoffeeScript, Apache Struts, and Perl experienced rapid community decline within five to ten years of peak adoption, often leaving organisations and developers with limited warning signals. Conversely, technologies such as Rust, TypeScript, and PostgreSQL have demonstrated sustained, measurable community growth, underpinned by strong enterprise adoption patterns and positive developer sentiment. This contrast illustrates that technology trajectory is, in principle, observable in advance through community and enterprise signals, provided those signals are systematically analysed rather than read informally.

The regional context sharpens the stakes further. In Kenya and the broader East African region, including Nairobi's "Silicon Savannah" fintech cluster and various government digital transformation initiatives, organisations frequently commit to technology stacks with limited access to localised, evidence-based adoption guidance. Developers in the region similarly depend on international trend publications that may not reflect local enterprise adoption realities or the specific technologies dominating regional employer demand.

This project builds directly on an existing open-source repository, developer-ecosystem-analytics (19 GitHub stars), which already aggregates six real-world datasets covering Stack Overflow community activity, developer sentiment survey responses, Fortune 500 enterprise technology stacks, company profiles, and technology metadata. The repository currently implements twelve descriptive SQL analytics queries and computes a composite Technology Health Score, but it offers no forward-looking, predictive capability. TechPulse is proposed as the predictive extension that closes this gap, transforming an existing, descriptive community-analytics tool into a forecasting system capable of classifying technologies as growing, stable, or declining.

Taken together, these observations frame technology-trajectory forecasting as both a well-motivated and a tractable problem: well-motivated because the financial and career stakes of getting technology-adoption decisions wrong are large and rising, and tractable because the developer-ecosystem-analytics repository already demonstrates that the underlying signal data, community activity, enterprise adoption depth, and developer sentiment, exists and is queryable. What is missing is not data, but a supervised model and an explainable presentation layer capable of turning that existing data into a forward-looking classification, which is precisely the contribution TechPulse sets out to make.

2. Problem Statement

Organisations and individual developers currently lack a reliable, data-driven method for anticipating whether a given software technology is heading toward growth, stability, or decline. Existing community analytics tools, including the developer-ecosystem-analytics repository this project extends, are descriptive rather than predictive: they report what has already occurred in a technology's community and enterprise adoption activity, but they do not forecast where that technology is heading next.

This limitation carries measurable consequences. Organisations that commit engineering resources to technologies on a silent decline trajectory risk costly, unplanned migrations. Developers who invest significant learning time in a technology approaching obsolescence face career setbacks that can take years to recover from. Yet current

decision-making remains largely dependent on informal opinion, trend articles, and developer surveys that describe present sentiment without projecting future trajectory.

The developer-ecosystem-analytics repository demonstrates that sufficient signal data already exists , six integrated datasets spanning community activity, enterprise adoption, and developer sentiment , to support forward-looking analysis. However, no predictive layer currently exists to translate these signals into actionable trajectory classifications. TechPulse addresses this gap directly by engineering predictive features from the existing signal data and classifying technologies as growing, stable, or declining using supervised machine learning.

3. Objectives

Main Objective

To develop TechPulse, a machine learning system that predicts the future trajectory (growth, stability, or decline) of software technologies using community activity, enterprise adoption, and developer sentiment signals drawn from the developer-ecosystem-analytics dataset corpus.

Specific Objectives

- To perform in-depth exploratory data analysis (EDA) on the existing six-dataset corpus to identify statistically significant patterns and correlations relevant to technology trajectory.
- To engineer predictive features from Stack Overflow activity trends, Fortune 500 company adoption depth, and developer sentiment survey scores.
- To train and evaluate machine learning classification models , Logistic Regression, K-Nearest Neighbours, Random Forest, and XGBoost , for technology trajectory prediction.
- To identify the strongest predictors of technology decline using SHAP (SHapley Additive exPlanations) feature-importance analysis.
- To design and deploy an interactive Streamlit dashboard that visualises technology predictions, risk scores, and feature-importance explanations in real time.
- To document the full project process, findings, and recommendations in a formal dissertation.

4. Significance of Project

For Developers

Individual developers will gain access to a data-driven, evidence-based tool for evaluating whether a technology they are considering learning, or deepening their expertise in, shows signs of growth, stability, or impending decline. This directly addresses the risk of career setbacks caused by over-investment in technologies that quietly lose community and enterprise support.

For Organisations

Technology decision-makers , including chief technology officers, engineering leads, and IT strategy teams , will benefit from an objective, explainable framework for assessing technology stack risk. TechPulse's risk scores and feature-importance explanations provide a structured basis for platform migration timing, technology refresh planning, and investment prioritisation.

For the Open-Source Community

Because TechPulse extends a publicly starred, open-source repository, the predictive layer and dashboard will be immediately available to the broader developer community upon release, transforming a static descriptive analytics tool into a living, predictive resource for the global technology community.

For Academic Research

This project contributes a novel combination of multi-signal feature engineering, explainable machine learning, and enterprise-scale adoption data to the technology lifecycle prediction literature , a gap not yet addressed by the existing studies reviewed in Chapter Two.

CHAPTER TWO: LITERATURE REVIEW AND REQUIREMENTS

2.1 Literature Review

Developer Ecosystem and Technology Adoption Analytics

Research into open-source and developer-community platforms has increasingly used community activity as a proxy for technology health. Studies of GitHub starring behaviour have demonstrated that social signals, including stars, forks, and contributor counts, reflect genuine interest and adoption patterns rather than arbitrary popularity, supporting the use of community metrics as predictive features (Borges & Valente, 2018). This finding is directly relevant to TechPulse's design choice to treat community signals such as Stack Overflow question volume as legitimate proxies for genuine developer interest rather than noise to be filtered out.

Work analysing the evolution of social coding platforms has similarly shown that contribution and activity patterns shift in measurable, trackable ways as projects and technologies mature, decline, or are superseded, directly motivating the trend-based features, such as Community Decay Rate and Growth Momentum Index, proposed for TechPulse (Mens & Goeminne, 2012). This body of work supports the underlying assumption of the featureengineering methodology (Proposal §3.2, SDS Module 2): that a technology's future trajectory leaves a measurable trace in its historical activity data well before that trajectory becomes obvious through informal channels such as trend articles or social media commentary.

Large-scale empirical studies linking programming-language choice to code-quality outcomes provide further evidence that the technology a team adopts has measurable downstream consequences, reinforcing the practical importance of forecasting technology trajectory rather than only describing its current state (Ray et al., 2014). This finding underpins the significance argument made in Chapter One: because technology choice has real, quantifiable consequences for code quality and organisational risk, a forecasting tool of the kind TechPulse proposes has practical value beyond simple academic curiosity.

Existing Approaches and Data Sources

The annual Stack Overflow Developer Survey is one of the most widely used public datasets for understanding developer sentiment, technology satisfaction, and adoption intent at scale (Stack Overflow, 2023/2024). It is descriptive by design: it reports current sentiment and usage but does not itself produce forward-looking predictions. TechPulse treats survey-derived sentiment as one input signal among several rather than as a standalone forecasting tool, since survey data captures developer intention and opinion but not actual enterprise deployment behaviour, a gap this project addresses by combining survey sentiment (via the Sentiment Delta feature) with Fortune 500 adoption data (via the Adoption Velocity and Company Diversity Score features), so that stated developer intent is checked against observed enterprise behaviour rather than taken at face value.

Empirical case studies of open-source project activity have shown that prior activity patterns, including commit frequency, issue resolution rates, and community responsiveness, carry predictive signal about a project's future trajectory (Syer et al., 2013). These studies, however, generally focus on individual projects rather than technologies as a whole, and do not combine community signals with enterprise adoption data. This single-project limitation is a gap that TechPulse directly addresses by aggregating signals across all technologies in the corpus, languages, frameworks, databases, and tools alike, rather than analysing individual repositories in isolation, which allows the resulting classifier to generalise across an entire technology category instead of a single codebase.

Gap Analysis and Justification for TechPulse

The literature reviewed above confirms three things: (a) community activity signals are valid proxies for technology health; (b) developer sentiment surveys provide useful but inherently descriptive adoption-intent data; and (c) empirical project-level studies demonstrate that activity patterns carry predictive signal. However, no existing work reviewed combines all three signal families, community activity, enterprise adoption depth, and developer sentiment, into a single supervised classification framework covering technology categories (languages, frameworks, databases, and tools) rather than individual repositories.

Unlike the Stack Overflow survey or single-project evolution studies described above, TechPulse combines these three distinct signal families into a single supervised classification framework. It also moves beyond description to prediction, explicitly labelling technologies as growing, stable, or declining and explaining those predictions through SHAP-based feature importance. This combination of multi-signal feature engineering, enterprise-scale adoption data, and explainable predictive modelling represents, to the author's knowledge, a novel contribution not addressed by the existing literature or tools reviewed above.

2.2 System Requirements Specifications

This section condenses the full System Requirements Specification (SRS) document produced for TechPulse, which formally defines the functional, non-functional, and external requirements guiding all design, development, and evaluation activity for STU 4101.

Purpose and Scope

The SRS serves as the authoritative reference for what TechPulse must accomplish, the constraints under which it must operate, and the quality standards it must meet. TechPulse WILL: ingest and process the six datasets already integrated in the developer-ecosystem-analytics PostgreSQL database without new data collection; perform comprehensive EDA across all six datasets, including correlation and distribution analysis; engineer seven predictive features (Technology Health Score, Growth Momentum Index, Question Quality Score, Company Diversity Score, Sentiment Delta, Adoption Velocity, and Community Decay Rate); assign trajectory labels using 12-month trailing data; train and evaluate four classification models with hyperparameter tuning; evaluate performance using accuracy, weighted F1-score, ROC-AUC, and confusion-matrix analysis with 5-fold crossvalidation; generate SHAP feature-importance values for the best-performing model; build and publicly deploy an interactive Streamlit dashboard; and document all findings in a formal dissertation.

TechPulse WILL NOT: collect new primary data; provide regulated financial or investment advice; guarantee prediction accuracy for technologies with insufficient historical signal data (these are flagged as low-confidence instead); monitor signals in real time (updates are delivered via scheduled batch processing); support multilanguage interfaces; process personal or private developer data; integrate paid third-party APIs beyond the Streamlit Cloud free tier; or train deep-learning models, since the project is scoped to classical supervised machine learning on structured tabular data. Expected outcomes include a model achieving at least 80% accuracy on the three-class task, a SHAP-derived ranking of decline predictors, a public dashboard covering more than 50 technologies, and a dissertation meeting all BSc. Data Science requirements.

System Perspective , Four Architectural Layers

TechPulse is not a standalone, greenfield system; it is a predictive extension of the existing developer-ecosystemanalytics repository, occupying the boundary between data engineering, machine learning research, and interactive software deployment. The SRS defines four conceptual layers:

- **Data Layer** , the six pre-integrated datasets stored in the existing PostgreSQL 14+ database; consumed but not structurally modified by TechPulse.
- **Feature Engineering and Modelling Layer** , a Python layer that reads from the database, computes the predictive feature matrix, assigns target labels, and trains/evaluates the four classification models.
- **Explainability Layer** , applies SHAP analysis to the best-performing trained model to derive featureimportance values and per-prediction explanations.
- **Presentation Layer** , the Streamlit dashboard, which loads the trained model and SHAP values and visualises risk scores, trajectory predictions, trend charts, and feature importance for each queried technology.

System Environment

Development and model training take place on a personal computer with a minimum of 8 GB RAM, a mid-range CPU (Intel Core i5 / AMD Ryzen 5 or better), and at least 50 GB of local storage; no GPU acceleration is required, as all four classifiers are CPU-bound. The production dashboard runs on Streamlit Cloud's shared infrastructure. The key components of the software environment are summarised below.

Category	Requirement
Programming language	Python 3.10+ (implementation); SQL (analytics queries)
Category	Requirement

Database	PostgreSQL 14+ (existing developer-ecosystem-analytics instance)
Core ML/analytics libraries	pandas, numpy, scikit-learn 1.3+, xgboost 1.7+, shap 0.42+
Visualisation & dashboard	matplotlib, seaborn, plotly 5.14+, streamlit 1.28+
Data access	sqlalchemy 2.0+, psycpg2 2.9+, joblib 1.3+ (model serialisation)
Version control	Git 2.x / GitHub
Cloud hosting	Streamlit Cloud (free tier); Heroku noted as fallback

Table 2.1 , Summary of the TechPulse Software Environment

User Characteristics

- Technology Decision-Makers (Primary Users) , CTOs, engineering managers, and enterprise architects who use the dashboard to support stack-selection and risk-management decisions; assumed to have general technical literacy but not deep ML expertise, so predictions must be presented in plain language.
- Individual Developers (Primary Users) , professionals or students deciding where to invest learning effort; comfortable with a web dashboard; primary goals are to identify growing/stable technologies worth specialising in and to understand which signals drive a given risk score.
- Academic Researchers and Practitioners (Secondary Users) , interact mainly through the open-source repository and dissertation; expected to have advanced ML familiarity and may replicate, validate, or extend the methodology.
- Project Supervisor (Administrative User) , Dr. Rufus Gireka, who reviews progress, dissertation chapters, and the final viva, and requires that the system remain correctly scoped and methodologically sound.

Condensed Functional Requirements

The full SRS specifies twenty functional requirements (FR-01 to FR-20), each documented with description, input, process, output, preconditions, postconditions, priority, exceptions, and validation/business rules. These are grouped below by functional area for this progress document.

Group	Requirements Covered	Summary
Data Pipeline	FR-01 – FR-04	Ingests and validates the six source datasets (null-rate and row-count checks), produces a full EDA report (distributions, correlations, timeseries trends, class balance), engineers the seven predictive features via SQL-query reuse and new trend calculations, and assigns each technology a Growing / Stable / Declining label based on Growth Momentum Index and 12-month question-volume trend.
Model Training	FR-05 – FR-10	Performs an 80/20 stratified train-test split with 5-fold cross-validation (FR-07), then trains and evaluates four classifiers , Logistic Regression (interpretable baseline), K-Nearest Neighbours (non-parametric baseline, k tuned via cross-validation), Random Forest and XGBoost (both hyperparameter-tuned via RandomizedSearchCV) , before producing a consolidated model-comparison report that selects the best model by weighted F1-score.
Explainability	FR-11	Applies a SHAP TreeExplainer (or LinearExplainer/KernelExplainer as a fallback) to the selected best model to produce a global top-7 featureimportance ranking and per-prediction SHAP values for a representative sample of technologies, stored for dashboard use.
Dashboard	FR-12 – FR-19	Delivers technology search and category filtering; a technology detail panel showing trajectory label, risk score (0–100), and confidence indicator; interactive trend charts (question volume, sentiment, adoption);

Group	Requirements Covered	Summary
		a SHAP feature-importance viewer with plain-English summary; an enterprise-adoption heatmap by sector; a global, sortable/exportable riskranking page; a model-performance summary page; and an About/documentation page with data-licence attributions and disclaimers.
Documentation	FR-20	Requires a formal dissertation covering Introduction, Literature Review, Methodology, Results, Discussion, and Recommendations, reviewed by the supervisor and submitted in bound form by the Semester 2 deadline.

Table 2.2 , Condensed Functional Requirements by Group

Candidate Models and Justification

The choice of four classifiers reflects a deliberate progression from interpretable baselines to higher-capacity models, so that the resulting dissertation can report on the accuracy/interpretability trade-off directly rather than assuming it.

Model	Justification	Evaluation Metrics
Logistic Regression	Interpretable baseline with coefficients that can be compared to SHAP values	Accuracy, F1-score (macro)
K-Nearest Neighbours	Non-parametric baseline; useful for detecting local clustering in feature space	Accuracy
Random Forest	Handles mixed feature types; provides native featureimportance scores	Accuracy, F1, ROC-AUC
XGBoost	State-of-the-art performance on tabular datasets; gradient boosting with regularisation	Accuracy, F1, ROC-AUC

Table 2.3 , Candidate Models, Justification, and Evaluation Metrics (Proposal §3.4)

Constraints

The SRS (§4.2.4) identifies seven categories of constraint that bound the design and delivery of TechPulse, all of which remain in force for the duration of the Final Year Project.

- **Technical Constraints** , limited to the six datasets already integrated in developer-ecosystem-analytics; no live external API access; the four classifiers (Logistic Regression, KNN, Random Forest, XGBoost) are fixed, with deep learning explicitly out of scope; the full training pipeline, including hyperparameter tuning, must complete in under two hours on the development hardware.
- **Budget Constraints** , the project operates under a tightly constrained budget (estimated at KShs 8,000 total, including printing and contingency; see Appendix II); all tools and hosting must be free or on a free tier.
- **Time Constraints** , the project must be completed within two academic semesters of 16 weeks each (32 weeks total), with every milestone in Appendix I met within that window.
- **Hardware Limitations** , model training is constrained to a personal computer with 8 GB RAM and a midrange CPU, limiting the size of hyperparameter search spaces; datasets exceeding available RAM must be processed in chunks or sub-sampled.
- **Software Limitations** , the Streamlit Cloud free tier imposes limits on concurrent users, memory allocation, and monthly active hours, which may degrade response times under high concurrent load; this is an accepted, documented constraint for the academic scope.
- **Legal and Ethical Constraints** , Stack Overflow data is used under the Stack Exchange Creative Commons Attribution-ShareAlike licence; Fortune 500 and company-profile data are used only in aggregate,

anonymised form; every prediction carries a confidence disclaimer to prevent misuse as a sole decision-making tool.

- Academic Project Constraints , the system must be developed and documented to a standard suitable for undergraduate final-year-project assessment, achievable by a single student developer within the defined timeline.

Use Case Summary

The SRS (§4.3.1) defines the system's actors and major use cases in UML-ready terms. Primary actors are the Technology Decision-Maker and Individual Developer (both interact with the public dashboard), and the Data Scientist/Project Developer, Tony Kenga, who executes the data pipeline, model-training routines, and deployment scripts. Secondary actors are the PostgreSQL database, Streamlit Cloud, and GitHub, each providing infrastructure rather than initiating use cases directly.

Use Case	Primary Actor(s)	Summary
UC-01 Search for Technology	Decision-Maker / Developer	Enter a technology name or category filter; system returns matching technologies with trajectory labels and risk scores.
UC-02 View Technology Prediction Detail	Decision-Maker / Developer	Select a technology; system displays trajectory prediction, risk score, confidence, trend charts, SHAP importance, and enterprise adoption data.
UC-03 Browse Global Risk Rankings	Decision-Maker / Developer	Navigate to the rankings page; system displays all technologies ranked by risk score with filters.
UC-04 Export Risk Rankings	Decision-Maker	Click download; system exports the current filtered ranking as CSV.
UC-05 View Model Performance	Decision-Maker / Researcher	Navigate to the model-performance page; system displays the four-model comparison table and best-model confusion matrix.
UC-06 Execute Data Pipeline	Data Scientist	Run the pipeline script; system ingests data, performs EDA, engineers features, assigns labels, trains and evaluates models, and runs SHAP analysis.
UC-07 Deploy Dashboard	Data Scientist	Push code to GitHub; Streamlit Cloud automatically redeploys the dashboard to the public URL.

Table 2.5 , Condensed Use Case Summary (SRS §4.3.1)

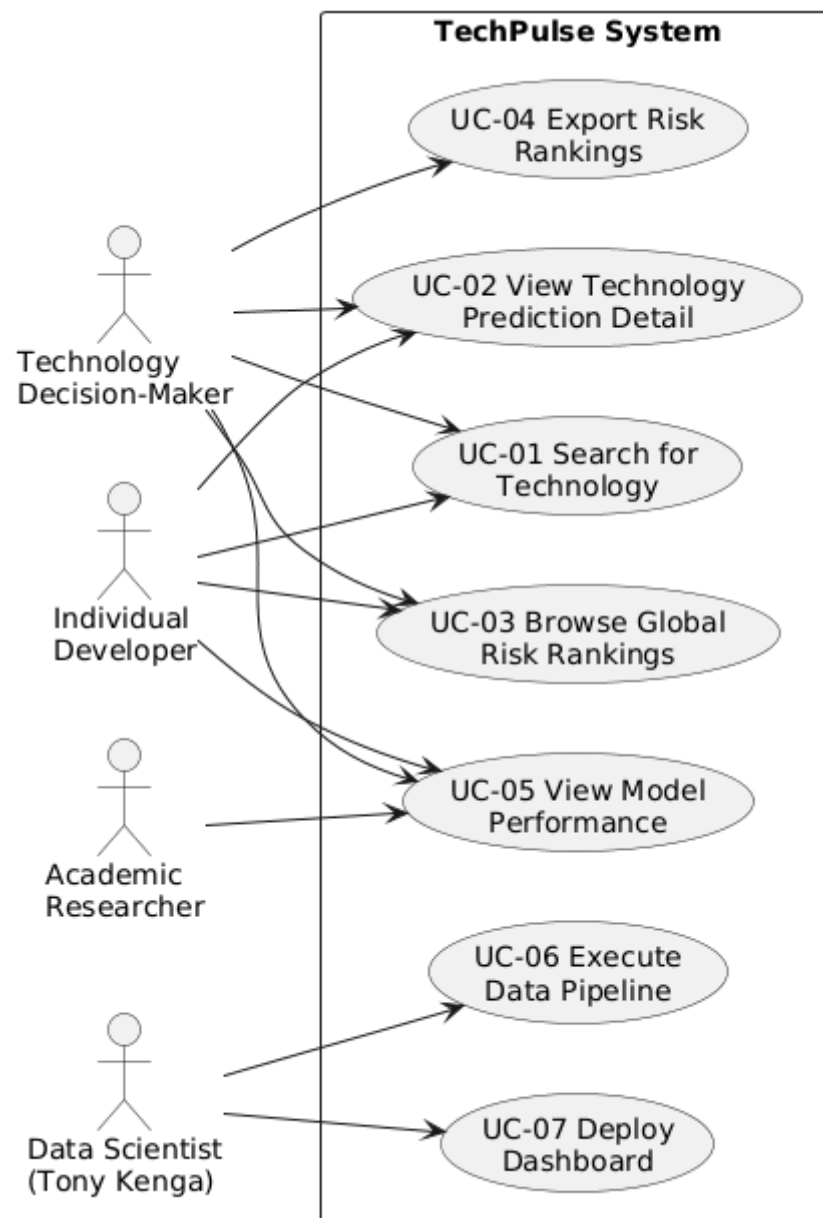


Figure 2.5.1 A Use case Diagram

Condensed Non-Functional Requirements

Alongside the twenty functional requirements, the SRS defines ten categories of non-functional requirement (NFR-P through NFR-AL) that establish the quality bar TechPulse must meet independent of any single feature. These are condensed below by category.

Category	Key Requirement(s)
Performance	Search/filter results within 1s (NFR-P-01); technology detail panel (prediction, trends, SHAP) renders within 2s, with on-demand SHAP capped at 10s (NFR-P-02); full modeltraining pipeline completes within 4 hours (NFR-P-03); data pipeline completes within 30 minutes (NFR-P-04).
Scalability	Pipeline must handle up to 1,000 technology records without architectural change; dashboard must serve up to 10 concurrent users on the Streamlit Cloud free tier.

Security	No user authentication for the public read-only dashboard; database credentials supplied only via environment variables, never hard-coded or committed; dependencies pinned and audited; HTTPS enforced by the hosting platform.
Reliability	Pipeline is idempotent via a fixed random_state = 42 across all stochastic operations; all steps use try-except error handling with detailed logging; each model file is versioned by a timestamp.
Usability	Non-expert accessibility , technical terms such as SHAP, F1-score, and ROC-AUC are accompanied by plain-English explanations; WCAG 2.1 AA colour contrast with text
Category	Key Requirement(s)
	labels alongside colour-coded trajectory badges; loading indicators for any operation exceeding one second.
Data Integrity & Privacy	Read-only access to source tables; deterministic, checksum-logged feature matrix; no personal developer data processed; Stack Overflow data used under its Creative Commons Attribution-ShareAlike licence, with attribution displayed on the dashboard; every prediction carries an informational-purposes-only disclaimer.
Auditability	Every pipeline run produces a timestamped execution log (step timings, row counts, null rates, random-state values, model-selection rationale); a JSON audit trail records hyperparameters, split ratios, and top-5 SHAP features for every training run.

Table 2.4 , Condensed Non-Functional Requirements

CHAPTER THREE: METHODOLOGY AND DESIGN

3.1 Data Collection Methods

TechPulse is a secondary-data study: it does not collect new primary data from human respondents, and no questionnaire, interview, or field survey is administered as part of this project. All required data is already integrated into the existing developer-ecosystem-analytics PostgreSQL database, in the form of six datasets: Stack Overflow Questions, Developer Sentiment Survey, Fortune 500 Tech Stacks, Company Profiles, Technology Metadata, and Question–Company Mapping. Because this infrastructure already exists, the "data collection" activity for TechPulse consists of correctly reusing and validating existing data rather than gathering new data from end users.

Three concrete processes substitute for primary data collection. First, the twelve existing SQL analytics queries in the developer-ecosystem-analytics repository are reviewed to identify which derived metrics, such as the Technology Health Score and Growth Momentum Index, can be reused or extended as machine-learning features rather than recomputed from scratch. Second, the project supervisor, Dr. Rufus Gireka, is consulted to validate the definition of the target variable (growing / stable / declining) and the model-evaluation criteria, since the labelling scheme is a project design decision rather than something present in the raw data. Third, the six integrated datasets are examined for completeness, class-balance, and feasibility of the intended feature engineering steps; this dataset-quality check is formalised in the SRS as FR-01 (Dataset Ingestion and Validation), which flags any key column exceeding a 5% null rate and any table containing fewer than 50 rows before feature engineering proceeds.

These approaches were selected in preference to new data collection because the necessary data infrastructure already exists: the research priority for TechPulse is to correctly translate existing descriptive metrics into a predictive feature set, not to gather requirements or opinions from a new population of end users. The specific dataset fields drawn upon by the feature-engineering pipeline are summarised in Appendix III, which functions as the closest equivalent to a data-collection instrument for this project.

3.2 Systems Design Specifications

This section condenses the full System Design Specification (SDS) document, which translates the objectives and requirements defined in the proposal and SRS into a concrete technical design covering architecture, database, program modules, interfaces, deployment, and security.

Purpose and Scope

The SDS describes the architecture, database, program modules, interfaces, deployment strategy, and security controls required to build TechPulse, and is intended to guide implementation across both semesters while serving as a technical reference for the supervisor and examiners. Its scope covers the feature-engineering pipeline, the four supervised classification models, the SHAP-based explainability engine, the PostgreSQL data and prediction store, and the Streamlit dashboard deployed on Streamlit Cloud. It explicitly excludes new primary data collection, financial/investment advisory logic, real-time monitoring, and mobile-native clients, consistent with the proposal's scope statement.

System Context and Assumptions

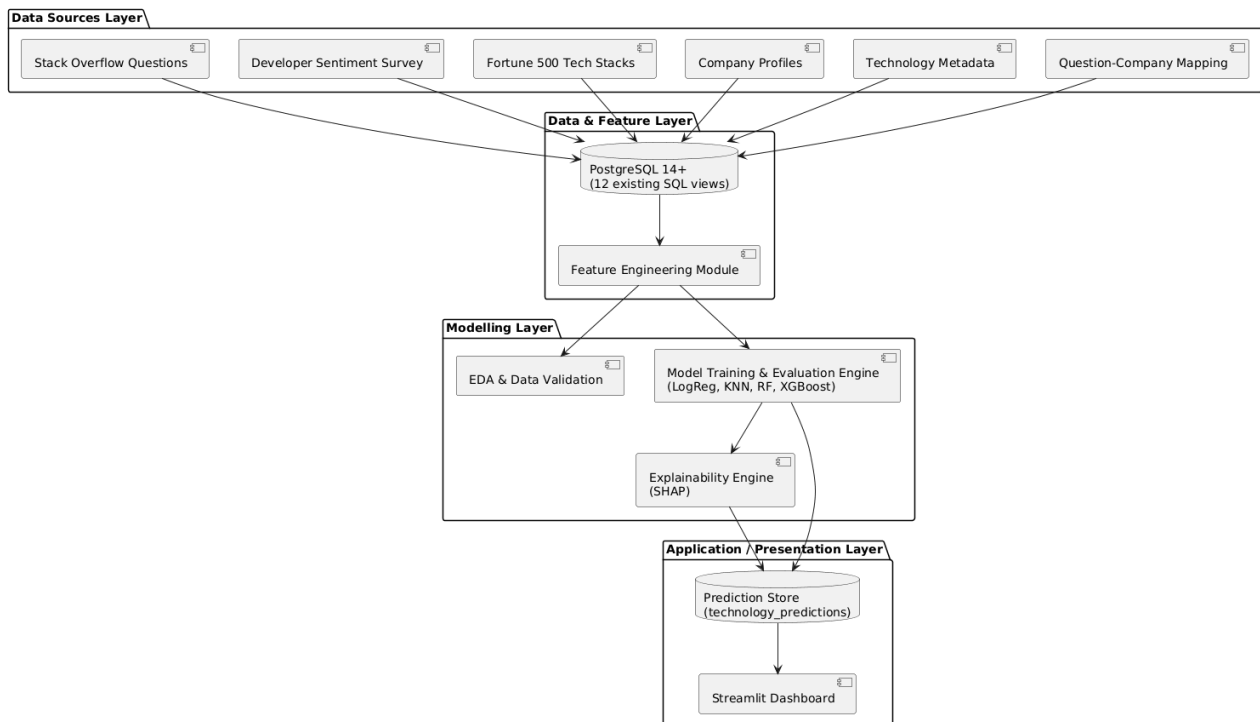
TechPulse sits between two existing boundaries: the six read-only datasets already integrated into the developer-ecosystem-analytics PostgreSQL database, and the end users, individual developers and organisational technology decision-makers, who consume trajectory predictions through a browser. TechPulse does not own or modify the source datasets; it reads from them, derives features, trains models offline, and writes predictions to a dedicated prediction store that the dashboard queries at request time. The design rests on five explicit assumptions and dependencies: that the six existing datasets remain accessible in the existing PostgreSQL 14+ instance throughout the project; that historical data volume is sufficient to construct a labelled training set, with technologies lacking sufficient history flagged as low-confidence rather than excluded outright; that Streamlit

Community Cloud free-tier hosting remains available, with a documented local fallback as mitigation; that Python 3.10+, scikit-learn, XGBoost, SHAP, and Streamlit remain compatible with one another, with version pins recorded in a requirements/lock file; and that no new personally identifiable data is collected, since all datasets are already anonymised or aggregated at the technology or company level.

Four-Layer Batch-Pipeline Architecture

TechPulse follows a four-layer, batch-pipeline architecture, illustrated in the SDS as Figure 5.1. At the base sits the

Data Sources Layer, the six existing datasets (Stack Overflow Questions, Developer Sentiment Survey, Fortune 500 Tech Stacks, Company Profiles, Technology Metadata, and Question–Company Mapping). These feed into the Data & Feature Layer, comprising the PostgreSQL 14+ database (with its twelve existing SQL analytics views) and the feature-engineering module that derives Community Decay Rate, Growth Momentum Index, Adoption Breadth, and Sentiment Trajectory. Above this sits the Modelling Layer, split into a Model Training & Evaluation Engine (Logistic Regression, KNN, Random Forest, XGBoost) running alongside an EDA & Data Validation module, and an Explainability Engine that ranks feature importance using SHAP. At the top is the Application/Presentation Layer, where a Prediction Store (the technology_predictions table) serves the Streamlit dashboard, which renders risk scores, trajectory charts, and SHAP views. Each layer communicates through welldefined tables or artefacts rather than direct in-memory coupling, which allows the modelling layer to be re-run independently of the dashboard.



3.2.1 Four-Layer Batch-Pipeline Architecture

Modules and Components

#	Module	Responsibility
1	Data Ingestion & Validation	Connects to PostgreSQL, extracts the six datasets, validates completeness and class balance.
2	Feature Engineering	Derives Community Decay Rate, Growth Momentum Index, Adoption Breadth, and Sentiment Trajectory features.

3	Model Training & Evaluation	Trains and cross-validates Logistic Regression, KNN, Random Forest, and XGBoost classifiers.
4	Explainability (SHAP)	Computes SHAP values for the best-performing model and ranks feature importance.
5	Prediction Store	Persists trajectory labels, risk scores, and SHAP summaries to PostgreSQL.
6	Dashboard / Presentation	Streamlit application rendering predictions, trends, and explanations to end users.

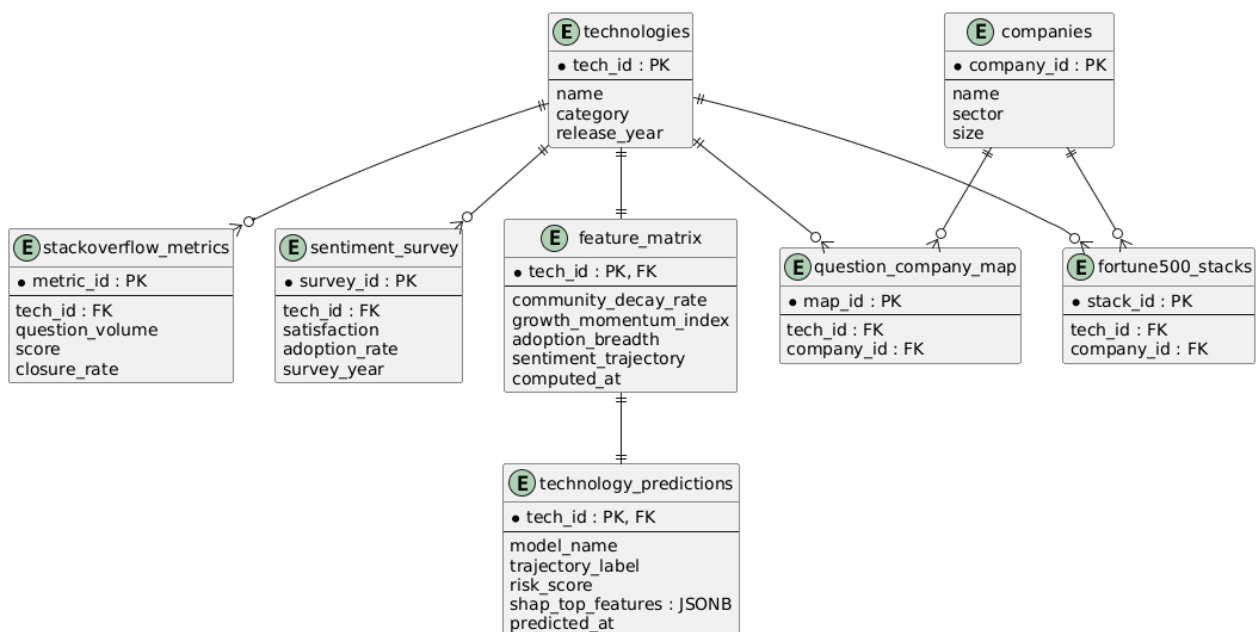
Table 3.1 , TechPulse Modules and Responsibilities (SDS §5.3.2)

Technology Stack

The frontend is built on Streamlit, with Plotly and Altair supplying interactive charts. The backend consists of Python 3.10+ batch-processing scripts and scheduled jobs handling ETL, feature engineering, and model training, with scikit-learn, XGBoost, and SHAP providing the modelling and explainability frameworks. PostgreSQL 14+ (the existing developer-ecosystem-analytics instance) is reused as the database management system, accessed through an internal SQLAlchemy-based data-access layer rather than any external paid API. Streamlit Community Cloud provides public hosting, with GitHub and GitHub Actions supplying version control, CI (linting, tests) and scheduled retraining jobs; Jupyter Notebooks and VS Code are the primary development tools.

Database Design

TechPulse extends the existing schema with two new tables , `feature_matrix` and `technology_predictions` , while reusing five existing entity tables (`technologies`, `stackoverflow_metrics`, `sentiment_survey`, `fortune500_stacks`, `companies`, `question_company_map`) as read-only sources, as summarised in the Entity Relationship Diagram (SDS Figure 5.2). The `feature_matrix` table stores one row per technology per computation run, versioned by a `computed_at` timestamp for reproducibility and rollback. The `technology_predictions` table stores the `trajectory_label`, a numeric `risk_score`, and a JSONB column summarising the top SHAP features for that prediction, so the dashboard can render explanations without recomputing SHAP at request time. Trained model artefacts (pickled scikit-learn/XGBoost objects) are stored on disk and referenced by a `model_version` identifier rather than stored inside the relational database.



3.2.2 Entity Relationship Diagram

The accompanying Level-0 Data Flow Diagram (SDS Figure 5.3) traces data from the external sources, through extraction and validation (process 0.1), feature engineering (0.2), model training/scoring (0.3), and finally to the published dashboard (0.4) consumed by end users, developers and CTOs alike. Data security measures include: aggregation at the technology or company level so that no individual developer personal data is stored or exposed; database credentials stored as environment variables or GitHub Secrets and never committed to source control; read-only database roles for the dashboard's query layer, with write access restricted to the scheduled batch job's service account; and TLS/SSL encryption for PostgreSQL connections where supported by the hosting provider.



3.2.3 Level-0 Data Flow Diagram

Program / Module Design

The SDS (§5.5.1) specifies the detailed process design for each of the five functional modules, including inputs, outputs, and processing logic. These are summarised below.

Module 1 , Data Ingestion & Validation

Connects to the existing PostgreSQL database via SQLAlchemy, extracts the six source tables (technologies, stackoverflow_metrics, sentiment_survey, fortune500_stacks, companies, question_company_map), and validates completeness, data types, and class distribution before any downstream processing begins. For each dataset, the module executes the extraction query, checks schema conformity, computes summary statistics, and raises a validation warning if a technology falls below the minimum data threshold defined in the proposal's risk mitigations. Output is a set of validated, in-memory DataFrames plus a data-quality report flagging missing values, duplicate records, and class imbalance.

Module 2 , Feature Engineering

Derives the predictive features used by all four classifiers from the validated datasets, reusing existing SQL analytics views where possible (e.g., the existing Technology Health Score) and computing new trend-based features. For each technology, the module computes a Community Decay Rate (rate of change in question volume and engagement over a rolling 12-month window), a Growth Momentum Index (ratio of recent 3-month activity to historical 24-month activity), Adoption Breadth (count and depth of Fortune 500 companies using the technology), and a Sentiment Trajectory Score (linear slope of satisfaction/adoption-intent across consecutive survey years). Output is a single feature_matrix table with one row per technology, persisted to the database and versioned by a computed_at timestamp.

Module 3 , Model Training & Evaluation

Trains and cross-validates the four candidate classifiers on the engineered feature matrix, using stratified sampling to handle class imbalance, and selects the best-performing model for production scoring. The module performs an 80/20 stratified train-test split, applies 5-fold cross-validation to each of Logistic Regression, K-Nearest Neighbours, Random Forest, and XGBoost, records per-class precision/recall/F1 and ROC-AUC for each, and persists the model with the highest macro-F1 score as the production model in a versioned model store.

Module 4 , Explainability (SHAP)

Computes SHAP values for the selected best-performing model to identify and rank the strongest predictors of technology decline, producing both global and per-technology explanations. An appropriate SHAP explainer is selected based on the model type, TreeExplainer for Random Forest or XGBoost, KernelExplainer or

LinearExplainer for Logistic Regression or KNN. SHAP values are computed for every technology; global feature importance is the mean absolute SHAP value per feature, and the top-N contributing features per technology are extracted and stored as JSON in the `technology_predictions.shap_top_features` column for dashboard rendering.

Module 5 , Prediction Store & Dashboard Rendering

Persists the final trajectory predictions and risk scores to the database and renders them, together with SHAP explanations, on the interactive Streamlit dashboard. Predictions are written to PostgreSQL in a single batch transaction (`tech_id`, `model_name`, `trajectory_label`, `risk_score`, `shap_top_features`, `predicted_at`). On each dashboard session, the application queries the `technology_predictions` table (cached with a one-hour time-to-live) and renders KPI cards, a trend chart, a sortable risk table, and a SHAP panel for the technology selected by the user.

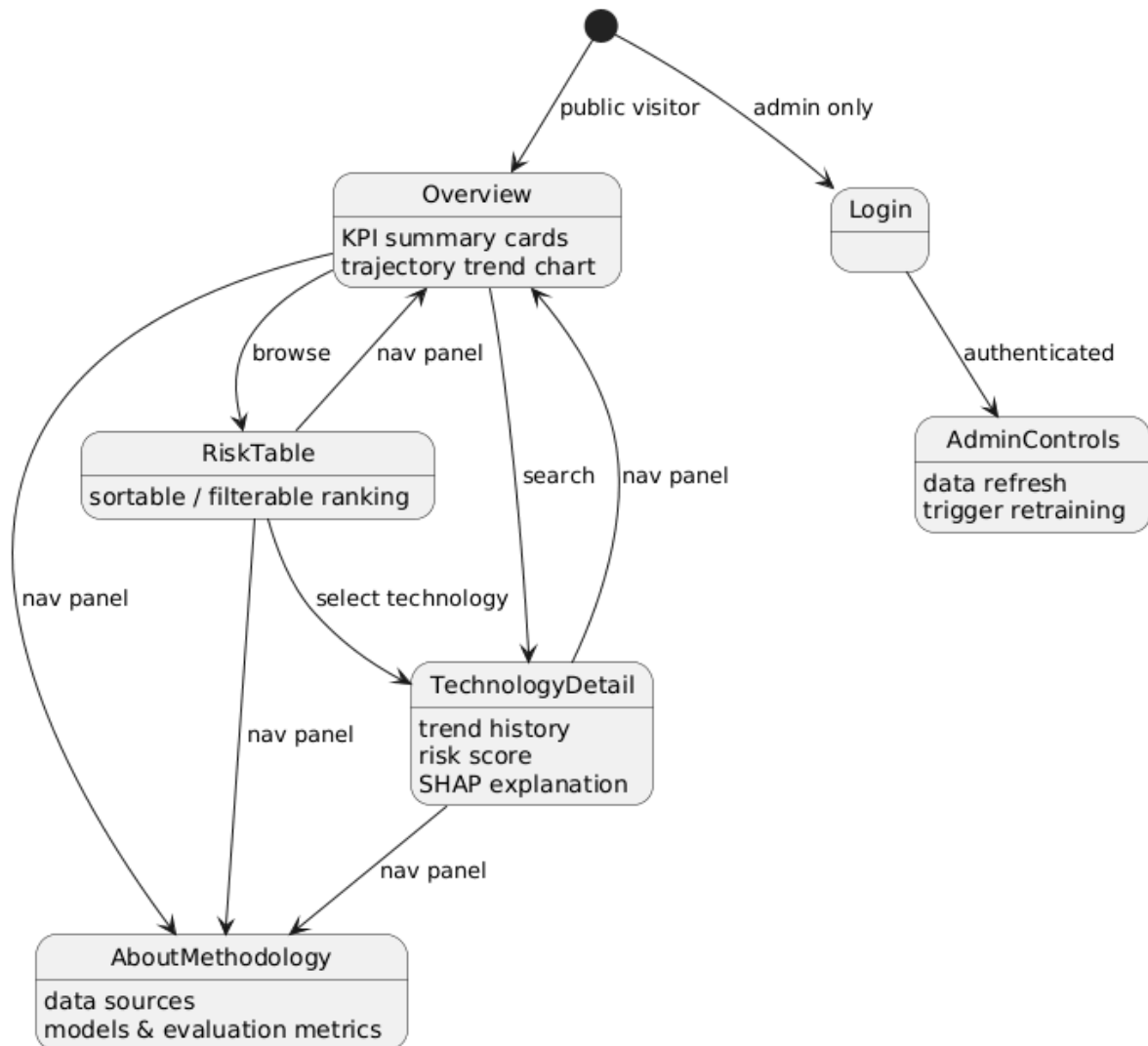
External and Hardware Interfaces

TechPulse does not depend on external third-party APIs for its core prediction functionality, since all required data already resides in the existing PostgreSQL database; an internal, SQLAlchemy-based data-access API mediates all reads and writes between the modelling pipeline, the prediction store, and the dashboard. No payment gateway is required, as TechPulse is a free, publicly accessible academic tool with no monetisation. Administrative access for triggering retraining uses a simple username/password login backed by hashed credentials stored in PostgreSQL; no third-party identity provider is required for this academic-scope deployment. The remaining third-party systems are GitHub, for source control and CI/CD triggers for scheduled retraining, and Streamlit Community Cloud, for application hosting of the public dashboard.

On the hardware side, the development computer requires a minimum Intel Core i5 or AMD Ryzen 5 processor (or equivalent), 8 GB RAM (16 GB recommended for comfortable hyperparameter tuning), and 50 GB of available disk space; no GPU, FPGA, or other specialised accelerator is required, since all four classifiers are CPU-bound. End users access the dashboard through a modern web browser (Chrome, Firefox, Safari, or Edge released within the past two years) on any internet-connected device, with the layout designed for desktop/laptop resolutions of 1280×720 or higher; mobile compatibility is a nice-to-have rather than a formal requirement. No specialised network hardware is required beyond a standard broadband connection of at least 10 Mbps for pushing code to GitHub, deploying to Streamlit Cloud, and accessing the deployed dashboard.

User Interface Design

The SDS specifies a lightweight administrative login screen (username/email, password, Sign In) that restricts access to data-refresh and retraining controls; public visitors browse the read-only dashboard without authentication. The main dashboard presents KPI summary cards (counts of growing/stable/declining technologies and average risk), a trajectory trend chart, a sortable/filterable risk-score table, and a SHAP feature-importance panel for the currently selected technology (Figure 5.4). The main functional screens are the Overview screen (aggregate KPIs and label distribution), the Technology Detail screen (per-technology trend history, risk score, and SHAP explanation), the Risk Table screen (sortable/filterable ranking by risk score and category), and the About/Methodology screen (data sources, models, and evaluation metrics for transparency). Navigation follows a simple flow: users land on the Overview screen, from which they can browse the Risk Table, search for a specific technology, and drill into its Technology Detail screen, with a persistent left-hand navigation panel accessible from anywhere in the application.

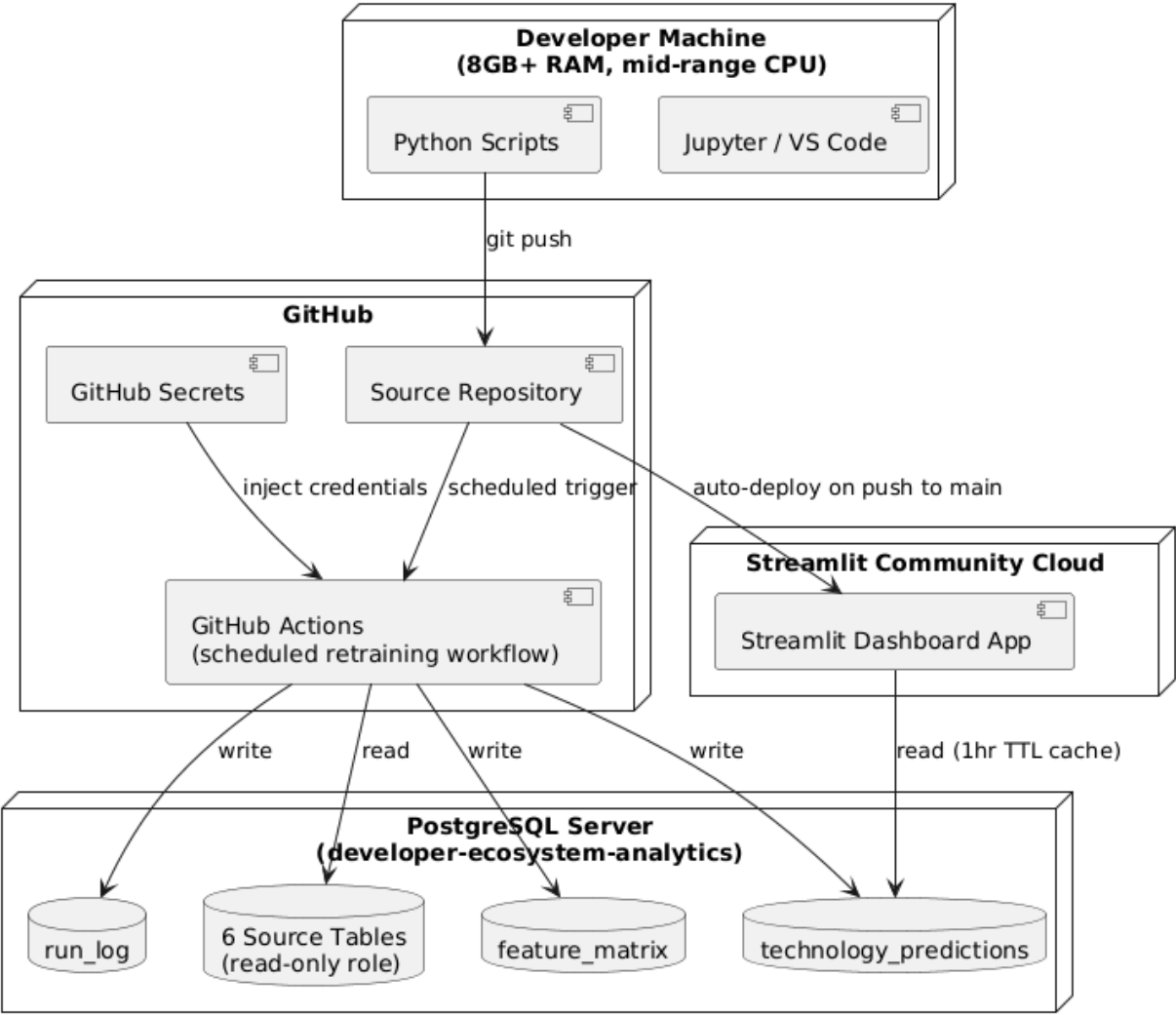


3.2.4 User Interface Design

Deployment and Security Design

Development occurs locally (8 GB+ RAM, mid-range CPU) with code pushed to the existing GitHub repository. Application deployment to Streamlit Community Cloud is automated on every push to the main branch, while model retraining is triggered on a scheduled batch basis via a GitHub Actions workflow rather than on every code push, reflecting the periodic nature of the underlying data (SDS Figure 5.5). Model artefacts are versioned via a `model_version` identifier; if a newly trained model underperforms on the held-out evaluation set, the previous production version remains active until the new model passes the evaluation gate, and application-code rollback uses standard Git revert/redeploy.

On the security side, public dashboard views remain read-only and unauthenticated, consistent with TechPulse's open-source, community-benefit goal, while administrative functions require authenticated login with bcrypthashed password storage and role-based database access that separates a read-only dashboard role from a readwrite batch-job service account. All secrets are stored as environment variables or GitHub Secrets and excluded from version control via `.gitignore`. Every batch pipeline run, ingestion, feature engineering, training, SHAP computation, and publishing, logs start/end timestamps, row counts, and validation warnings to a `run_log` table, and model evaluation metrics for every training run are retained (not overwritten) to support historical comparison and drift detection, with GitHub Actions CI logs providing a further audit trail of code changes and deployment events.



3.2.5 Deployment and Security Design

3.3 Test Plan

The test plan for TechPulse is derived directly from the evaluation strategy defined in the proposal (§3.5) and the functional and non-functional requirements defined in the SRS. It spans four testing categories: model evaluation, data validation, functional/dashboard testing, and reliability/reproducibility testing. Together, these categories test the system along the two dimensions that matter most for an explainable predictive tool of this kind, whether its predictions are statistically sound, and whether the system delivering those predictions behaves correctly, quickly, and reproducibly for its end users.

Model evaluation testing follows an 80/20 stratified train-test split with a fixed `random_state` of 42 to ensure the class balance across Growing, Stable, and Declining is preserved in both partitions, combined with 5-fold stratified cross-validation on the training split only, so that no information from the held-out test set leaks into model selection or hyperparameter tuning. Evaluation is not limited to overall accuracy: weighted F1-score is used as the primary model-selection metric specifically because it penalises poor performance on minority classes, and macroaveraged ROC-AUC and per-class confusion matrices are computed to surface any class-specific weaknesses that a single aggregate accuracy figure would hide.

Test Type	What Is Verified	Method / Tooling	Acceptance Criterion
Model Evaluation	Predictive performance of each of the four classifiers on unseen data	80/20 stratified train-test split (<code>random_state = 42</code>); 5-fold stratified cross-validation; accuracy, weighted F1-score, macro-averaged ROC-AUC, and per-class confusion matrices	Best-performing model reaches the target of at least 80% accuracy on the three-class task; weighted F1-score is the primary model-selection criterion
Data Validation	Completeness and reliability of the six source datasets prior to feature engineering	Automated null-rate checks (flag if a key column exceeds 5% nulls), minimum row-count checks (flag if a table has fewer than 50 rows), and class-imbalance checks against the trajectory label	All six tables pass validation with warnings logged; technologies below the minimum data threshold are flagged as low-confidence rather than silently included
Functional / Dashboard	Correct behaviour and responsiveness of each dashboard feature (search/filter, prediction + risk score + confidence display, trend charts, SHAP viewer, enterprise adoption heatmap, global rankings + CSV export, model-performance page)	Manual and scripted UI testing against each Functional Requirement (FR-12–FR-19); timing measurements against the NFR performance targets	Search/filter returns results within 1s; technology detail panel renders within 2s; ondemand SHAP computation completes within 10s with a loading indicator shown

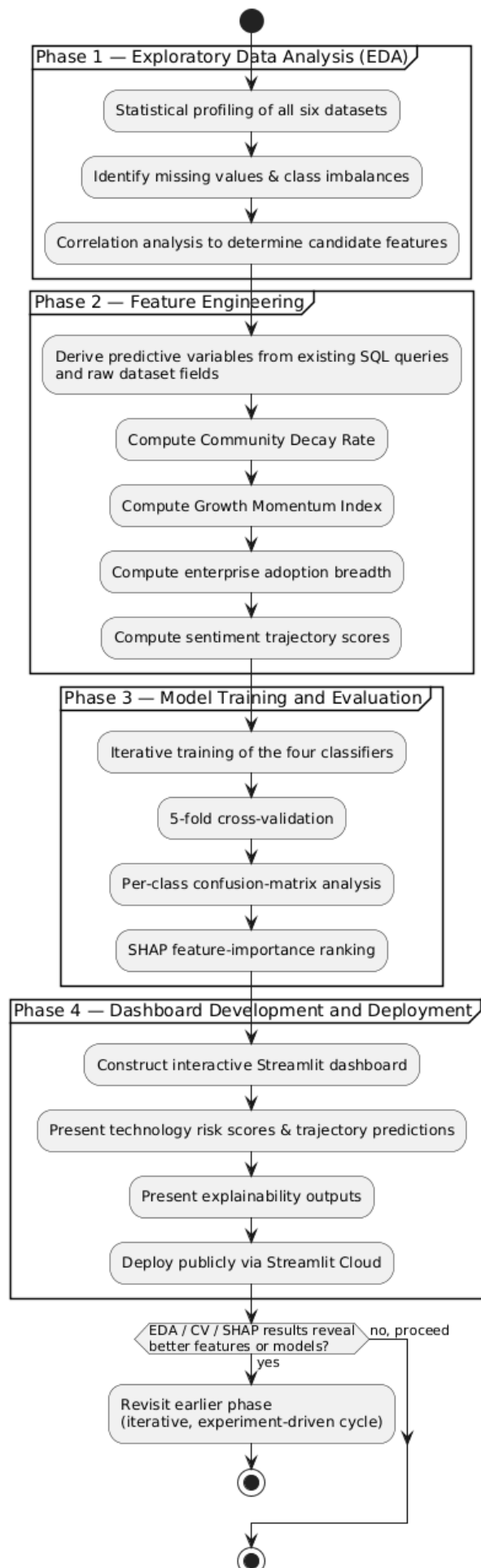
Reliability / Reproducibility	Deterministic, idempotent pipeline behaviour and correct error handling	Re-running the pipeline on identical input data and comparing output feature-matrix checksums (MD5); verifying that all stochastic steps use the fixed random_state = 42; inspecting execution logs for complete step-level detail	Identical inputs always produce an identical feature matrix and identical labels; all errors are logged with step name, error type, message, and stack trace rather than silently swallowed
-------------------------------	---	--	---

Table 3.2 , TechPulse Test Plan Summary

3.4 Implementation Plan

TechPulse follows an iterative, experiment-driven development methodology suited to machine-learning research, rather than a traditional software-only lifecycle such as Waterfall. Implementation proceeds through four iterative phases, each of which is revisited as EDA, cross-validation, and SHAP results reveal which features and models perform best , similar in spirit to Agile's short, evaluative cycles but adapted for a data-science workflow.

- Phase 1 , Exploratory Data Analysis (EDA): statistical profiling of all six datasets, identification of missing values and class imbalances, and correlation analysis to determine candidate features.
- Phase 2 , Feature Engineering: derivation of predictive variables from existing SQL queries and raw dataset fields, including Community Decay Rate, Growth Momentum Index, enterprise adoption breadth, and sentiment trajectory scores.
- Phase 3 , Model Training and Evaluation: iterative training of the four classifiers with 5-fold crossvalidation, per-class confusion-matrix analysis, and SHAP feature-importance ranking.
- Phase 4 , Dashboard Development and Deployment: construction of the interactive Streamlit dashboard presenting technology risk scores, trajectory predictions, and explainability outputs, deployed publicly via Streamlit Cloud.



3.4.1 Implementation phases activity diagram

At the schedule level, the project is organised across two 16-week academic semesters (32 weeks total; the full week-by-week breakdown is reproduced in Appendix I). Semester 1 is dedicated to data, EDA, and modelling: it opens with a literature review and annotated bibliography (Weeks 1–2), proceeds through deep EDA and statistical correlation analysis (Weeks 3–4), feature engineering into a feature matrix (Weeks 5–6), baseline model training for Logistic Regression and KNN (Weeks 7–9), advanced model training and hyperparameter tuning for Random Forest and XGBoost (Weeks 10–12), model evaluation and SHAP analysis (Weeks 13–14), and closes with a Semester 1 progress report and supervisor review (Weeks 15–16). Semester 2 is dedicated to the dashboard, dissertation, and presentation: it opens with building and deploying the Streamlit dashboard (Weeks 1–5), continues through drafting dissertation chapters one through five (Weeks 6–11), proceeds through full dissertation review, correction, and formatting (Weeks 12–13), and closes with final submission (Week 14) and presentation/viva preparation (Weeks 15–16). This phased approach keeps the four iterative development phases aligned to concrete, examinable deliverables at each stage.

Risk Management

Implementation planning for TechPulse incorporates the risk register defined in the proposal (Appendix D), which identifies five risks to on-time, on-scope delivery and the corresponding mitigation built into the implementation plan.

Risk	Effect	Mitigation
Class imbalance in growing/stable/declining labels	Biased model performance; misleading accuracy	Stratified sampling; report F1-score and ROC-AUC alongside accuracy
Insufficient historical signal for newer technologies	Unreliable predictions for some categories	Flag low-confidence predictions; exclude technologies below a minimum data threshold
Hardware failure or theft	Delay; lost code/data	Regular commits to GitHub; periodic backups to flash disk/cloud storage
Dashboard hosting downtime or free-tier limits	Dashboard unavailable for evaluation	Maintain a local fallback build; document an alternative hosting option
Scope creep beyond defined project boundaries	Missed deadlines	Refer back to the defined project scope; defer extras to future work

Table 3.3 , Project Risks and Mitigations (Proposal Appendix D)

REFERENCES

- Borges, H., & Valente, M. T. (2018). What's in a GitHub star? Understanding repository starring practices in a social coding platform. *Journal of Systems and Software*, 146, 112–129. <https://doi.org/10.1016/j.jss.2018.09.016>
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). ACM. <https://doi.org/10.1145/2939672.2939785>
- Gartner. (2023). Gartner forecasts worldwide IT spending to grow 4.3% in 2023. Gartner, Inc. <https://www.gartner.com/en/newsroom/press-releases/2023-01-18-gartner-forecasts-worldwide-it-spending-to-grow-4-percent-in-2023>
- IEEE Computer Society. (2011). IEEE standard glossary of software engineering terminology (IEEE Std 610.121990, reaffirmed 2002). IEEE. <https://standards.ieee.org/>
- ISO/IEC/IEEE 42010:2011. Systems and software engineering , Architecture description. International Organization for Standardization.
- Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems* (Vol. 30, pp. 4765–4774). Curran Associates.
- Mens, T., & Goeminne, M. (2012). Analysing the evolution of social coding platforms. *Science of Computer Programming*, 78(8), 1034–1053. <https://doi.org/10.1016/j.scico.2012.04.008>
- OWASP Foundation. OWASP Top Ten. Used as a baseline reference for the TechPulse security design.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. PostgreSQL Global Development Group. PostgreSQL 14 documentation. <https://www.postgresql.org/docs/14/>
- Python Software Foundation. Python 3.10 documentation. <https://docs.python.org/3.10/>
- Ray, B., Posnett, D., Filkov, V., & Devanbu, P. (2014). A large scale study of programming languages and code quality in GitHub. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering* (pp. 155–165). ACM. <https://doi.org/10.1145/2635868.2635922> scikit-learn developers. Scikit-learn user guide. https://scikit-learn.org/stable/user_guide.html
- SHAP developers. SHAP documentation. <https://shap.readthedocs.io/>

Stack Exchange, Inc. (2023). Stack Overflow developer survey 2023. <https://survey.stackoverflow.co/2023/> Stack

Exchange, Inc. (2024). Stack Overflow developer survey 2024. <https://survey.stackoverflow.co/2024/>

Streamlit Inc. (2023). Streamlit documentation (Version 1.28). <https://docs.streamlit.io/>

Syer, M. D., Adams, B., Hassan, A. E., & Godfrey, M. W. (2013). Revisiting prior empirical findings for agile: An analysis of the RapidSMS open-source project. In Proceedings of the 2013 Conference of the Centre for Advanced Studies on Collaborative Research (pp. 49–65). IBM Corp.

XGBoost developers. XGBoost documentation. <https://xgboost.readthedocs.io/>

Wieggers, K., & Beatty, J. (2013). Software requirements (3rd ed.). Microsoft Press.

APPENDICES

Appendix I: Research Schedule

Semester 1 , Data, EDA and Modelling

Week(s)	Activity	Deliverable
1–2	Literature review: technology lifecycle prediction, ML on developer data	Annotated bibliography
3–4	Deep EDA on all six datasets; statistical correlation analysis	EDA report and visualisations
5–6	Feature engineering: derive all ML features from existing SQL queries	Feature matrix (CSV)
7–9	Train baseline models: Logistic Regression and KNN	Baseline model results
10–12	Train advanced models: Random Forest and XGBoost; hyperparameter tuning	Final trained models
13–14	Model evaluation, SHAP analysis, feature importance ranking	Evaluation report
15–16	Semester 1 progress report and supervisor review	Progress report

Table A.1 , Semester 1 Research Schedule

Semester 2 , Dashboard, Dissertation and Presentation

Week(s)	Activity	Deliverable
1–3	Build Streamlit dashboard: tech risk scores, predictions, trend charts	Live dashboard
4–5	Deploy dashboard (Streamlit Cloud)	Public URL
6–8	Write dissertation chapters 1–3: Introduction, Literature, Methodology	Draft chapters
9–11	Write dissertation chapters 4–5: Results, Discussion and Recommendations	Draft chapters
12–13	Full dissertation review, corrections, formatting	Complete draft
14	Final submission	Bound dissertation
15–16	Presentation preparation and viva	Presentation slides

Table A.2 , Semester 2 Research Schedule

Appendix II: Project Budget

SN	Resource	Quantity	Availability	Estimated (KShs)	Actual (KShs)
1	Internet connection	40 hours	Own	3,000	
2	Cloud/dashboard hosting (Streamlit Cloud)	2 semesters	Free tier	0	
3	Flash disk / external backup	1	Own	1,500	
4	Computer	1	Own	0 (already owned)	
5	Printing & binding (dissertation)	2 copies	To purchase	2,500	
6	Contingency	,	,	1,000	

Table A.3 , TechPulse Project Budget (Proposal Appendix B)

Appendix III: Data Collection Instrument (e.g. Questionnaire)

TechPulse does not employ a questionnaire, interview schedule, or other primary-data collection instrument. All analytical data is drawn from six datasets already integrated into the existing developer-ecosystem-analytics PostgreSQL database, so no instrument was administered to human respondents for this project. In place of a questionnaire, the table below reproduces the Data Sources specification from the proposal (Section 3.3), which defines precisely what data is drawn from each existing dataset and functions as the closest equivalent to a datacollection instrument for a secondary-data study of this kind.

Dataset	Source	Key Features Used
Stack Overflow Questions	Existing DB	Volume, scores, answers, closure rate by tag
Developer Sentiment Survey	Existing DB	Satisfaction, adoption rate, learning curve
Fortune 500 Tech Stacks	Existing DB	Company adoption depth per technology
Company Profiles	Existing DB	Parent company relationships, size, sector
Technology Metadata	Existing DB	Category, release year, lifecycle flags
Question–Company Mapping	Existing DB	Technology mentions by company over time

Table A.4 , Data Sources Used in Place of a Primary Data-Collection Instrument